

*Hiord[#]: An Approach to the Specification and Verification of Higher-Order (C)LP Programs**

MARCO CICCALÈ, DANIEL JURJO-RIVAS and JOSE F. MORALES

Universidad Politécnica de Madrid (UPM), IMDEA Software Institute, Madrid, Spain

(e-mails: m.ciccale@alumnos.upm.es, marco.ciccale@imdea.org, daniel.jurjo@alumnos.upm.es,
daniel.jurjo@imdea.org, josefrancisco.morales@upm.es, josef.morales@imdea.org)

PEDRO LÓPEZ-GARCÍA

Spanish Council for Scientific Research, IMDEA Software Institute, Madrid, Spain

(e-mails: pedro.lopez@csic.es, pedro.lopez@imdea.org)

MANUEL V. HERMENEGILDO

Universidad Politécnica de Madrid (UPM), IMDEA Software Institute, Madrid, Spain

(e-mails: manuel.hermenegildo@upm.es, manuel.hermenegildo@imdea.org)

submitted 22 July 2025; revised 22 July 2025; accepted 27 July 2025

Abstract

Higher-order constructs enable more expressive and concise code by allowing procedures to be parameterized by other procedures. Assertions allow expressing partial program specifications, which can be verified either at compile time (statically) or run time (dynamically). In higher-order programs, assertions can also describe higher-order arguments. While in the context of (constraint) logic programming ((C)LP), run-time verification of higher-order assertions has received some attention, compile-time verification remains relatively unexplored. We propose a novel approach for statically verifying higher-order (C)LP programs with higher-order assertions. Although we use the Ciao assertion language for illustration, our approach is quite general and we believe is applicable to similar contexts. Higher-order arguments are described using predicate properties – a special kind of property which exploits the (Ciao) assertion language. We refine the syntax and semantics of these properties and introduce an abstract criterion to determine conformance to a predicate property at compile time, based on a semantic order relation comparing the predicate property with the predicate assertions. We then show how to handle these properties using an abstract interpretation-based static analyzer for programs with first-order assertions by reducing predicate properties to first-order properties. Finally, we report on a prototype implementation and evaluate it through various examples within the Ciao system.

KEYWORDS: higher-order, static analysis, assertions, abstract interpretation, (constraint) logic programming.

* Partially funded by MICIU projects CEX2024-001471-M *María de Maeztu* and TED2021-132464B-I00 *PRODIGY*, as well as by the Tezos foundation. We would also like to thank the anonymous reviewers for their very useful and constructive feedback.

1 Introduction

Abstraction is a fundamental principle in computer science often used for managing complexity. Higher-order constructs are a form of abstraction that enables writing code that is more concise and expressive by allowing procedures to be parameterized by other procedures, resulting in more modular and maintainable code. (Constraint) logic programming languages like Prolog (Körner et al. 2022) and functional programming languages like Haskell (Marlow 2010) have included different forms of higher-order since their early days, and languages from other programming paradigms like Java or C++ have adopted them later on. In particular, Prolog systems allow defining higher-order predicates and making higher-order calls. For example, the query: `?- filter(even,[7,4,9],L)`, passes the term `even` as an argument to the higher-order predicate `filter/3`, which *applies* the `even/1` predicate to each element of the input list, selecting those that succeed, yielding `L = [4]`. Assertions are linguistic constructs for writing partial program specifications, which can then be verified or used to detect deviations in program behavior w.r.t. such specifications. The *assertion-based approach* to program verification (Hermenegildo et al. 1999; Puebla et al. 2000b; Sanchez-Ordaz et al. 2021) differs from other approaches such as strong type systems (Cardelli 1989) in that assertions are optional and can include properties that are undecidable at compile time, and thus some checking may need to be relegated to run time. Hence, the assertion-based approach is closer to gradual typing in functional languages (Siek and Taha 2006). The combination of higher-order predicates and assertions in the (C)LP context was already explored by Stulova et al. (2014). This work introduced the notion of *predicate properties*, a special kind of properties that allow using the full power of the (Ciao) assertion language for describing the higher-order arguments of procedures. This work also proposed an operational semantics for *dynamically* checking higher-order (C)LP programs annotated with such higher-order assertions. However, the *static* verification of programs with higher-order assertions was not addressed in that work, and remains relatively unexplored since other related work in (C)LP that supports higher order (*e.g.*, Miller (1991); Hill and Lloyd (1994); Somogyi et al. (1996)) generally adheres to the strong typing model. In this work we propose a novel approach for the compile-time verification of higher-order (C)LP programs with assertions describing higher-order arguments. We present a refinement of both the syntax and the semantics of predicate properties (§3). Next, we define an abstract criterion to determine whether a predicate conforms to a predicate property at compile time, based on a semantic order relation between the definition of a predicate property and the partial specification of a predicate (§4.1). Then, we introduce an approach for “casting” predicate usage in a program analysis-friendly manner that enhances and complements the proposed abstract criterion (§4.2). We also propose a technique for dealing with these properties using an abstract interpretation-based static analyzer for programs with first-order assertions, by representing predicate properties as first-order properties that are natively understood by such an analyzer (§4.3). Finally, we present a prototype implementation of these techniques and study its application to a number of examples (§5). For concreteness, we use in our presentation the Ciao (Hermenegildo et al. 2012) assertion language, and make use of its CiaoPP preprocessor (Hermenegildo et al. 2005), that combines both static and dynamic analysis. However, we believe the approach is quite general and flexible, and can be applied, at least conceptually, to similar gradual approaches.

2 Preliminaries and notation

Variables start with a capital letter. The set of terms is inductively defined as follows: (1) variables are terms (2) if f is an n -ary function symbol and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term. We use the overbar notation $(\bar{\cdot})$ to denote a finite sequence of elements (e.g., $\bar{t} \equiv t_1, \dots, t_n$), and write $|\bar{\cdot}|$ for representing its length. An *atom* has the form $p(\bar{t})$ where p is an n -ary predicate symbol, and $\bar{t}$ are terms. The function $\text{ar}(p)$ denotes the arity of a predicate p . A *higher-order atom* has the form $X(\bar{t})$ where X is a variable and $\bar{t}$ are terms. (Note that variables are *not* allowed in the function symbol position of terms, only in literals). A *constraint* is a conjunction of expressions built from predefined predicates whose arguments are constructed using predefined functions and variables, for example, $X - Y > \text{abs}(Z)$. A *literal* is either an atom, a higher-order atom, or a constraint. *Negation* is encoded as finite failure, supported through a program expansion. A *goal* is a finite sequence of literals. A *rule* has the form $H :- B$ where H , the *head*, is an atom and B , the *body*, is a possibly empty finite sequence of literals. A *higher-order constraint logic program*, or *higher-order program* P is a finite set of rules. We use σ to represent a variable renaming, and $\sigma(L)$ or $L\sigma$ to represent the result of applying σ to a syntactic object L . The *definition* of an atom L in a program, $\text{defn}(L)$, is the set of renamed program rules s.t. each renamed rule has L as its head. We assume that all rule heads are *normalized*, that is, H is an atom of the form $p(\bar{v})$ where $\bar{v}$ are distinct variables. Let $\bar{\exists}_L \theta$ denote the projection of the constraint θ onto the variables of L . We denote *constraint entailment* by $\theta_1 \models \theta_2$.

2.1 Operational semantics of higher-order programs

The operational semantics of a higher-order program P is given in terms of its *derivations*, which are sequences of *reductions* between *states*. A state $\langle G | \theta \rangle$ consists of a goal G , and a constraint store (or store) θ . We denote sequence concatenation by $(::)$. We assume for simplicity that the underlying constraint solver is complete and projection exists. We use $S \rightsquigarrow S'$ to indicate that a reduction step can be applied to state S to obtain state S' . Naturally, $S \rightsquigarrow^* S'$ indicates that there is a sequence of reduction steps from S to S' . A state $S = \langle L :: G | \theta \rangle$ where L is a literal, is *reduced* to a state S' as follows:

1. If L is a constraint and $\theta \wedge L$ is satisfiable, then $S' = \langle G | \theta \wedge L \rangle$.
2. If L is an atom and $\exists(L :- B) \in \text{defn}(L)$, then $S' = \langle B :: G | \theta \rangle$.
3. If L is a higher-order atom of the form $X(\bar{t})$, then $S' = \langle p(\bar{t}) :: G | \theta \rangle$ given that $\exists p \in P. \theta \models (X = p) \wedge \text{ar}(p) = |\bar{t}|$.

Let L be an atom, $S = \langle L :: G | \theta \rangle$, $S' = \langle G | \theta' \rangle$, and suppose $S \rightsquigarrow^* S'$. We refer to S as a *call state* for L , and S' as a *success state* for L . A *query* Q is a pair (L, θ) , where L is a literal and θ a store for which the (C)LP system starts a computation from state $\langle L | \theta \rangle$. The set of all derivations of P from a query Q is denoted $\text{derivs}(P, Q)$, and this notation is naturally extended to a set of queries $\mathcal{Q}$. Let $D_{[-1]}$ denote the last state of any derivation D . A finite derivation from a query Q is *finished* if the last state in the derivation cannot be reduced. A finished derivation from a query Q is *successful* if the last state is of the form $\langle \square | \theta' \rangle$, where $\square$ denotes the empty goal sequence. In that case, the constraint $\bar{\exists}_L \theta'$ is an *answer* to Q . We denote by $\text{answers}(P, Q)$ the set of answers of

P to a query Q . A finished derivation is *failed* if the last state is not of the form $\langle \square \mid \theta \rangle$. A query Q *finitely fails* if all derivations in $\text{derivs}(P, Q)$ are finished and have failed.

2.2 Property formulas

Conditions on the constraint store are stated as *property formulas*. A property formula is a DNF formula of *property literals*. A property literal is a literal corresponding to a special kind of predicates called *properties*. Properties are typically defined in the source language, in the same way as ordinary predicates but marked accordingly, and are required to meet certain conditions (Hermenegildo et al. 1999; Puebla et al. 2000b). In particular, they are normally required to be checkable at run time but not necessarily decidable at compile time, where they are safely approximated.¹

Example 2.1 (Properties).

The following program defines the properties **list**/1 (“being a list”) and **prefix**/2 (“being a prefix of a list”):

```

1 | :- prop list/1. list([]).           3 | :- prop prefix/2. prefix([], Ys) :- list(Ys).
2 | list([_ | Xs]) :- list(Xs).       4 | prefix([X | Xs], [X | Ys]) :- prefix(Xs, Ys).
```

The property formula $(\text{list}(Xs), \text{list}(Ys), \text{prefix}(Xs, Ys))$ states that Xs and Ys should be lists, and that Xs should be a prefix of Ys . This formula contains three property literals corresponding to the **list**/1 and **prefix**/2 properties.

We now recall an instrumental definition about properties from Puebla et al. (2000b):

Definition 2.1 (Succeeds trivially).

A property literal L succeeds trivially for θ in a program P , denoted $\theta \Rightarrow_P L$, iff $\exists \theta' \in \text{answers}(P, (L, \theta))$. $\theta \models \theta'$. A property formula succeeds trivially for θ if all of the property literals of at least one conjunct of the formula succeeds trivially.

Intuitively, a property literal (or formula) succeeds trivially if it succeeds for θ in P without adding new “relevant” constraints to θ . For example, **list**(X) checks “ X being a list.”

2.3 Traditional assertions

Assertions are syntactic objects for expressing properties of programs that must be satisfied at program execution. We recall the herein relevant parts of the assertion schema of Puebla et al. (2000a). *Traditional* (or *first-order*) *predicate* (or *pred*) assertions have the following syntax: “:- **pred** $Pred$: $Pre \Rightarrow Post$.” where $Pred$ is a normalized atom representing a predicate, and Pre and $Post$ are property formulas. They express that all calls to $Pred$ must satisfy the pre-condition Pre , and, if such calls succeed, the post-condition $Post$ must be satisfied. If there are several *pred* assertions, the Pre field of at least one of them must be satisfied.

¹ Ciao assertions can also include *global properties*, which may not always be checkable at run time (e.g., termination), but we focus for brevity on the described types of assertions and properties.

Example 2.2 (Assertions).

The following assertions for the `take/3` predicate relating a list and its prefix:

```

1 | :- pred take(N,Xs,Ys) : (int(N), list(Xs)) => prefix(Ys,Xs).
2 | :- pred take(N,Xs,Ys) : (list(Xs), prefix(Ys,Xs)) => int(N).

```

restrict the meaning of `take/3` as follows:

- `take(N,Xs,Ys)` must be called with `Xs` bound to a list, and either `N` bound to an integer or `Ys` bound to a prefix of `Xs`.
- if `take(N,Xs,Ys)` succeeds when called with `N` bound to an integer and `Xs` bound to a list, then `Ys` must be bound to a prefix of `Xs`.
- if `take(N,Xs,Ys)` succeeds when called with `Xs` bound to a list and `Ys` bound to a prefix of `Xs`, then `N` must be bound to an integer.

We represent checks on the store by a set of assertions with a set of *assertion conditions*.

Definition 2.2 (Assertion conditions).

Given a predicate represented by a normalized atom $Pred$, and its corresponding set of assertions $\{A_1, \dots, A_n\}$ with $A_i = \text{“:- pred } Pred : Pre_i \Rightarrow Post_i.\text{”}$, the set of assertion conditions for $Pred$ is $\{C_0, C_1, \dots, C_n\}$ with

$$C_i = \begin{cases} \text{calls}(Pred, \bigvee_{j=1}^n Pre_j) & i = 0 \\ \text{success}(Pred, Pre_i, Post_i) & i \in 1..n \end{cases}$$

Condition C_0 encodes the checks that ensure that all calls to the predicate represented by $Pred$ are within those admissible by the set of assertions; we refer to it as the *calls assertion condition*. Conditions $C_1, \dots, C_n$ encode the checks for compliance of the successes for particular sets of calls, and we call them the *success assertion conditions*.

From this point on, we denote by $\mathcal{A}$ both the set of assertions of the program and, interchangeably, its associated set of assertion conditions. Also, for a normalized atom $Pred$, $\mathcal{A}(Pred)$ denotes only the assertions of $\mathcal{A}$ associated to the predicate $Pred$.

Example 2.3 (Assertion conditions).

The set of assertion conditions for the set of pred assertions in Example 2.2 is:

```

calls(take(N,Xs,Ys), (int(N), list(Xs)) ∨ (list(Xs), prefix(Ys,Xs)))
success(take(N,Xs,Ys), (int(N), list(Xs)), prefix(Ys,Xs))
success(take(N,Xs,Ys), (list(Xs), prefix(Ys,Xs)), int(N))

```

2.4 Operational semantics of higher-order programs with traditional assertions

This operational semantics checks whether assertion conditions hold or not while computing the derivations from a query, halting the derivation as soon as an assertion condition is violated. For identifying a possible assertion condition violation, every assertion condition C is related to a unique label ℓ via a mapping $\text{label}(C) = \ell$. States of derivations are now of the form $\langle G \mid \theta \mid \mathcal{E} \rangle$, where $\mathcal{E}$ denotes the set of labels for falsified assertion conditions (with $|\mathcal{E}| \leq 1$); while such set is unnecessary if execution halts upon an assertion condition violation, we include it to keep the semantics presented in this paper close to

that of previous work. A finished derivation from a query $Q = (L, \theta)$ is now *successful* if the last state is of the form $\langle \Box \mid \theta' \mid \emptyset \rangle$, *failed* if the last state is of the form $\langle L' \mid \theta' \mid \emptyset \rangle$, and *erroneous* if the last state is of the form $\langle L' \mid \theta' \mid \{\ell\} \rangle$. We also extend the set of literals with syntactic objects of the form $\text{check}(L, \ell)$ where L is a literal and ℓ is a label for an assertion condition, which we call *check literals*. Thus, a *literal* is now a constraint, an atom, a higher-order atom, or a check literal. We now recall the notion of *Semantics with Assertions* from Stulova et al. (2018), which we adapt to support higher-order atoms. A state $S = \langle L :: G \mid \theta \mid \emptyset \rangle$, can be *reduced* to a state S' , denoted $S \rightsquigarrow_{\mathcal{A}} S'$, as follows:

1. If L is a constraint or a higher-order atom, then $S' = \langle G' \mid \theta' \mid \emptyset \rangle$, with G' and θ' obtained as in the operational semantics without assertions: $\langle L :: G \mid \theta \rangle \rightsquigarrow \langle G' \mid \theta' \rangle$.
2. If L is an atom and $\exists(L :- B) \in \text{defn}(L)$, then

$$S' = \begin{cases} \langle G \mid \theta \mid \{\ell\} \rangle & \text{if } \exists C = \text{calls}(L, \text{Pre}) \in \mathcal{A}. \text{label}(C) = \ell \wedge \theta \not\vdash_P \text{Pre} \\ \langle B :: \text{Post}C :: G \mid \theta \mid \emptyset \rangle & \text{otherwise} \end{cases}$$

where $\text{Post}C = \text{check}(L, \ell_1) :: \dots :: \text{check}(L, \ell_n)$ includes all the checks $\text{check}(L, \ell_i)$ such that $\ell_i = \text{label}(C_i)$, with $C_i = \text{success}(L, \text{Pre}_i, \text{Post}_i) \in \mathcal{A} \wedge \theta \Rightarrow_P \text{Pre}_i$.

3. If L is a check literal $\text{check}(L', \ell)$, then

$$S' = \begin{cases} \langle G \mid \theta \mid \{\ell\} \rangle & \text{if } \exists C = \text{success}(L, _, \text{Post}) \in \mathcal{A}. \text{label}(C) = \ell \wedge \theta \not\vdash_P \text{Post} \\ \langle G \mid \theta \mid \emptyset \rangle & \text{otherwise} \end{cases}$$

The set of derivations for a program P with assertions $\mathcal{A}$ from a set of queries $\mathcal{Q}$ using the semantics with assertions is denoted $\text{deriv}_{\mathcal{A}}(P, \mathcal{Q})$. Given a predicate represented by a normalized atom L , a store θ , and a set of queries $\mathcal{Q}$, we define the *success context* $\mathcal{S}_{\mathcal{A}}(L, \theta, P, \mathcal{Q})$ of L and θ for P and $\mathcal{Q}$ as $\{\exists_L \theta' \mid \exists D \in \text{deriv}_{\mathcal{A}}(P, \mathcal{Q}). \exists G. \langle L :: G \mid \theta \rangle \in D. D_{[-1]} = \langle G \mid \theta' \rangle\}$. Intuitively, the *success context* of a predicate p with its assertions is the set of stores of the success states of p obtained using the semantics above.

2.5 Static program analysis by abstract interpretation

Abstract interpretation (Cousot and Cousot 1977) is a mathematical framework for constructing sound, static program analysis tools. These tools extract properties about a program by interpreting it over a special *abstract domain* ($D^\sharp$), whose elements are finite representations of (possibly infinite) sets of actual constraints in the *concrete domain* (D). Elements of the concrete and abstract domains are related by two functions: *abstraction* ($\alpha : D \rightarrow D^\sharp$) and *concretization* ($\gamma : D^\sharp \rightarrow D$). Provided certain conditions on $D^\sharp$, α , and γ , abstract interpretation guarantees soundness and termination of the analysis.

2.6 Goal-dependent abstract interpretation

We use, for concreteness, *goal-dependent abstract interpretation*, in particular the PLAI algorithm (Muthukumar and Hermenegildo 1992). This technique takes a program P , an abstract domain $D^\sharp$, and a set of initial abstract queries $\mathcal{Q}^\sharp$, describing all the possible initial concrete queries to P . An abstract query $Q^\sharp$ is a pair (L, λ) , where L is an atom and $\lambda \in D^\sharp$ an abstraction of a set of concrete initial program states (e.g., constraint stores). A set of abstract queries $\mathcal{Q}^\sharp$ represents a set of concrete queries defined

as $\gamma(Q^\#) = \{(L, \theta) \mid (L, \lambda) \in Q^\# \wedge \theta \in \gamma(\lambda)\}$. The algorithm computes a set of triples $\{\langle L_1, \lambda_1^c, \lambda_1^s \rangle, \dots, \langle L_n, \lambda_n^c, \lambda_n^s \rangle\}$ where L_i is an atom, and λ_i^c and λ_i^s are abstractions approximating the set of all call and success states for L_i , respectively, for all occurrences of literal L_i in all possible derivations of P from $\gamma(Q^\#)$. Higher-order atoms are supported by reducing them to first-order calls when the called predicate can be determined by the analysis, or making conservative assumptions otherwise. For the rest of the paper, we assume that the abstract interpretation of a program P for the set of initial abstract queries $Q^\#$, denoted by $\llbracket P \rrbracket_{Q^\#}^\#$, works with an implicit abstract domain $D^\#$, which safely approximates the concrete values and operations. Although not strictly required, $D^\#$ has a lattice structure with a bottom-most element $\perp$, meet ($\sqcap$), join ($\sqcup$), and less than ($\sqsubseteq$) operators. As usual, $\perp$ denotes the abstraction s.t. $\gamma(\perp) = \emptyset$.

2.7 Compile-time verification of (first-order) assertions.

In addition to generating the results mentioned above, the analyzer also checks any (first-order) assertions in the program by safely approximating the property formulas of such assertions, and comparing them against the analysis results ($\llbracket P \rrbracket_{Q^\#}^\#$) using the abstract operators.² The verification result is reported as changes in the status and transformations of the assertions: **checked** if the properties are satisfied; **false** if some property is proved not to hold; or **check** if neither of the first two can be determined, in which case run-time checks will be inserted into the program to ensure run-time safety. The verification process yields an assignment of a value **checked**, **false**, or **check** to each assertion in $\mathcal{A}$, denoted $\text{acheck}(\mathcal{A}, \llbracket P \rrbracket_{Q^\#}^\#)$.

3 Specifying higher-order programs: predicate properties

In higher-order (C)LP, variables can be bound to predicate symbols that are later invoked. This naturally gives rise to the need for expressing conditions on these predicates that must hold during program execution. To this end, *predicate properties* were introduced in Stulova et al. (2014), which we revise and refine here.³ A predicate property is defined as a set of *anonymous assertions*. Anonymous assertions generalize traditional assertions by allowing the predicate symbol in the *Pred* field to act as a placeholder.

Definition 3.1 (Anonymous assertion).

An anonymous assertion $\mathcal{A}$ is an assertion whose *Pred* field is of the form $_(\bar{v})$, where $\bar{v}$ are free, distinct variables, and $_$ is a placeholder for a predicate symbol.⁴ Instantiating $_$ with a specific predicate symbol p produces a traditional assertion for p derived from the anonymous assertion $\mathcal{A}$, denoted $\mathcal{A}|_p$.

² We refer the reader to Puebla et al. (2000b); Sanchez-Ordaz et al. (2021) for the technical details on this subject.

³ We propose a more compact syntax here that avoids having to use a named variable for the anonymous predicate symbol (as in Stulova et al. (2014)) and takes advantage of functional notation ($:=$).

⁴ We also use for compactness “ $_$ ” as anonymous functor, a syntactic extension from the Ciao hiord package (Cabeza et al. 2004), but double quotes “ $_$ ” can also be used to stay within ISO-Prolog syntax.

Example 3.1 (Anonymous assertion).

Let $\mathcal{A}$ be the anonymous assertion: “**:- pred** $_ (X,Y) : \text{int}(X) \Rightarrow \text{int}(Y) ..$ ” Then $\mathcal{A}|_p$ is the traditional assertion: “**:- pred** $p(X,Y) : \text{int}(X) \Rightarrow \text{int}(Y) ..$ ” obtained by instantiating the anonymous assertion $\mathcal{A}$ with the predicate symbol p .

Definition 3.2 (Predicate property).

A predicate property Π is a set of anonymous assertions $\{\mathcal{A}_1, \dots, \mathcal{A}_n\}$. Its syntax is: “ $\Pi := \{ \text{:- pred } _ (\bar{v}) : Pre_1 \Rightarrow Post_1. \dots \text{:- pred } _ (\bar{v}) : Pre_n \Rightarrow Post_n. \}$.” The function $\text{ar}(\Pi)$ denotes the arity of the predicates for which all of the anonymous assertions in Π express a property. Instantiating Π with a specific predicate symbol p produces a set of traditional assertions for p , denoted $\Pi|_p = \{\mathcal{A}_1|_p, \dots, \mathcal{A}_n|_p\}$.

We use Π to refer to both a set of anonymous assertions and, interchangeably, the corresponding set of anonymous assertion conditions; extending *instantiation* accordingly.

Example 3.2 (Predicate property).

The following program defines the predicate property **int_op** (“being a predicate that behaves as an integer nondeterministic binary operator”), and a higher-order assertion:

```

1 | int_op := { :- pred  $\_ (X,Y,Z) : (\text{int}(X), \text{int}(Y)) \Rightarrow \text{int}(Z) .$  }.
2 | :- pred  $\text{eval}(A,B,\text{Op},R) : (\text{int}(A), \text{int}(B), \text{int\_op}(\text{Op})) \Rightarrow \text{int}(R) .$ 

```

The predicate property literal **int_op**(Op) states that Op should be a 3-ary predicate s.t., if called with its first two arguments bound to integers, then its third argument should be bound to an integer upon success. The higher-order assertion for the **eval**/4 predicate states that it must be called with its first two arguments bound to integers and its third argument bound to a predicate that conforms to property **int_op**, and that if any such call succeeds, then its fourth argument should be bound to an integer. $\square$

4 Verifying higher-order programs

Once established how to specify higher-order programs using predicate properties, we now concentrate on how to verify such programs. We first recall some instrumental definitions from Puebla et al. (2000b) for reasoning about abstractions of property formulas. For the rest of the discussion, let P be a program and F a property formula defined in P .

Definition 4.1 (Trivial success set).

We define the trivial success set of F as $F^\natural = \{\bar{\exists}_F \theta \mid \theta \Rightarrow_P F\}$.

Example 4.1 (Trivial success set).

Let $F = \text{list}(\mathcal{L})$, both $\theta_1 = \{\mathcal{L} = [1,2]\}$ and $\theta_2 = \{\mathcal{L} = [1,X]\}$ are in $F^\natural$, but $\theta_3 = \{\mathcal{L} = [1|_]\}$ is not, since a call to $(\mathcal{L} = [1|_], \text{list}(\mathcal{L}))$ would further instantiate the second argument of $[1|_]$. The trivial success set $F^\natural$ of F captures the notion that the **list**($\mathcal{L}$) property formula requires $\mathcal{L}$ to be instantiated to (the structure of) a list.

Definition 4.2 (Abstract trivial success subset).

An abstraction is an abstract trivial success subset of F , denoted $F^{\sharp-}$, iff $\gamma(F^{\sharp-}) \subseteq F^\natural$.

Definition 4.3 (Abstract trivial success superset).

An abstraction is an abstract trivial success superset of F , denoted $F^{\sharp+}$, iff $\gamma(F^{\sharp+}) \supseteq F^{\natural}$.

Intuitively, $F^{\sharp-}$ and $F^{\sharp+}$ are, respectively, a safe under- and over-approximation of the trivial success set $F^{\natural}$ of the property formula F , and they can always be computed at compile-time by choosing the closest element in the abstract domain.

4.1 Conformance to a predicate property

When we provide a partial specification for a higher-order argument X of a higher-order predicate using a predicate property Π , we are describing requirements that the predicates that X may be bound to must meet. We refer to a predicate p behaving correctly w.r.t. Π as: p *conforming* to Π . We will now formalize this notion, with the goal of being able to safely approximate the set of predicates that X can be bound to without violating Π .

Definition 4.4 (Covered predicate).

Given $C = \text{calls}(_(\bar{v}), {}^{\circ}\text{Pre}) \in \Pi$ and ${}^{\circ}C = \text{calls}(p(\bar{v}), \text{Pre}) \in \mathcal{A}$, we say that p can be covered with Π iff ${}^{\circ}\text{Pre}^{\natural} \subseteq \text{Pre}^{\natural}$.

Intuitively, p can be covered with Π if the set of admissible calls to p is a superset of the set of admissible calls described by Π .

Definition 4.5 (Redundance).

Under the same conditions as in Definition 4.4, given that p can be covered with Π , we define the set of assertion conditions $\mathcal{A}'$ as follows:

$$\mathcal{A}' = \{\text{calls}(p(\bar{v}), \text{Pre} \wedge {}^{\circ}\text{Pre})\} \cup (\mathcal{A} \setminus \{C\}) \cup (\Pi \setminus \{{}^{\circ}C\})|_p$$

Given a sequence of literals G , let $\mathcal{U}(G)$ denote the result of removing all check literals from G . We extend $\mathcal{U}$ to derivations so that $\mathcal{U}(D)$ denotes the derivation resulting from transforming any (extended) state $\langle G \mid \theta \mid \mathcal{E} \rangle$ in D into the state $\langle G' \mid \theta \rangle$, where $G' = \mathcal{U}(G)$. Let Q_p be a query to p . We say that Π is *redundant* for p under Q_p iff

$$\forall D' \in \text{derivs}_{\mathcal{A}'}(P, Q_p). D'_{[-1]} = \langle G' \mid \theta \mid \{\ell'\} \rangle,$$

and

$$\forall D \in \text{derivs}_{\mathcal{A}}(P, Q_p). \mathcal{U}(D) = \mathcal{U}(D'),$$

it holds that $D'_{[-1]} \rightsquigarrow_{\mathcal{A}}^* \langle G \mid \theta \mid \{\ell\} \rangle$ through a derivation that reduces only check literals (if any at all),⁵ where ℓ (resp., ℓ') is the label for a **calls** or **success** assertion condition in $\mathcal{A}(p(\bar{v}))$ (resp., $\mathcal{A}'(p(\bar{v}))$).

Intuitively, a predicate property Π is redundant for a predicate p under a query Q_p to p iff augmenting the original set of assertion conditions ($\mathcal{A}$) with that of Π ($\mathcal{A}'$) does not introduce new run-time check errors in any derivation starting from Q_p .⁶

⁵ Note that this implies $\mathcal{U}(G) = \mathcal{U}(G')$.

⁶ Note that, for the purposes of determining conformance, the assertions for the predicates in the program can be provided by the user, inferred by analysis, or a combination of both.

Definition 4.6 (Conformance).

Let $\mathcal{Q}_p$ be the set of all possible queries to p . A predicate p conforms to Π , denoted $p \prec \Pi$, iff $\forall Q_p \in \mathcal{Q}_p. \Pi$ is redundant for p under Q_p . Conversely, p does not conform to Π , denoted $p \not\prec \Pi$, iff $\exists Q_p \in \mathcal{Q}_p. \Pi$ is not redundant for p under Q_p .

To prove that a predicate conforms to a predicate property, all possible derivations from all possible queries to that predicate have to be considered, which is often not feasible in practice. To this end, we introduce the notion of *abstract conformance* as a compile-time conformance criterion. Abstract conformance safely approximates the notion of conformance by comparing the assertion conditions of a predicate and those of a predicate property under the order relation of an abstract domain. We denote by $(\prec^{\sharp-})$ the notion of *strong* abstract conformance, and by $(\prec^{\sharp+})$ that of *weak* abstract conformance. That is, an under- and over-approximation of abstract conformance, respectively. Intuitively, strong abstract conformance captures only predicates known to conform, while weak abstract conformance also includes those for which conformance is unknown. Thus, the negation of weak abstract conformance captures the predicates that are known not to conform.

Definition 4.7 (Abstract conformance on “calls”).

Let Pre be the pre-condition of the **calls** assertion condition for p in $\mathcal{A}$, and ${}^\circ C$ be an anonymous **calls** assertion condition $\text{calls}(_(\bar{v}), {}^\circ \text{Pre})$. Then:

$$\begin{aligned} p \prec^{\sharp-} {}^\circ C &\Leftrightarrow (\text{Pre}^{\sharp+} \sqsubseteq {}^\circ \text{Pre}^{\sharp-}) \wedge (\text{Pre}^{\sharp-} \sqsupseteq {}^\circ \text{Pre}^{\sharp+}) \\ p \not\prec^{\sharp+} {}^\circ C &\Leftrightarrow \text{Pre}^{\sharp+} \sqcap {}^\circ \text{Pre}^{\sharp+} = \perp \end{aligned}$$

Definition 4.8 (Abstract conformance on “success”).

Let $\mathcal{A}$ be the set of assertion conditions for p , and ${}^\circ C$ be an anonymous **success** assertion condition $\text{success}(_(\bar{v}), {}^\circ \text{Pre}, {}^\circ \text{Post})$. Then:

$$\begin{aligned} p \prec^{\sharp-} {}^\circ C &\Leftrightarrow \exists S \subset \mathcal{A}, (\text{Pre}_{\sqcup}^{\sharp-} \sqsupseteq {}^\circ \text{Pre}^{\sharp+}) \wedge (\text{Post}_{\sqcup}^{\sharp+} \sqsubseteq {}^\circ \text{Post}^{\sharp-}), \\ \text{where } \begin{cases} \text{Pre}_{\sqcup}^{\sharp-} &= \sqcup \{ \text{Pre}^{\sharp-} \mid \text{success}(p(\bar{v}), \text{Pre}, _) \in S \} \\ \text{Post}_{\sqcup}^{\sharp+} &= \sqcup \{ \text{Post}^{\sharp+} \mid \text{success}(p(\bar{v}), _, \text{Post}) \in S \} \end{cases} \\ p \not\prec^{\sharp+} {}^\circ C &\Leftrightarrow \exists \text{success}(p(\bar{v}), \text{Pre}, \text{Post}) \in \mathcal{A}, (\text{Pre}^{\sharp+} \sqsubseteq {}^\circ \text{Pre}^{\sharp-}) \wedge \\ &\quad \wedge (\text{Post}^{\sharp+} \sqcap {}^\circ \text{Post}^{\sharp+} = \perp) \wedge \exists \theta \in \text{Pre}^{\sharp}. \mathcal{S}_{\mathcal{A}}(p(\bar{v}), \theta, P, \gamma(\mathcal{Q}_p^{\sharp})) \neq \emptyset \end{aligned}$$

where $\mathcal{Q}_p^{\sharp}$ is the set of abstract queries s.t. $\gamma(\mathcal{Q}_p^{\sharp})$ is a superset of the set of all valid queries to p described by the **calls** assertion condition of p in $\mathcal{A}$.

Definition 4.9 (Abstract conformance).

We define abstract conformance to a predicate property as follows:

$$\begin{aligned} p \prec^{\sharp-} \Pi &\Leftrightarrow \forall {}^\circ C \in \Pi_C. p \prec^{\sharp-} {}^\circ C \\ p \not\prec^{\sharp+} \Pi &\Leftrightarrow \exists {}^\circ C \in \Pi_C. p \not\prec^{\sharp+} {}^\circ C \end{aligned}$$

Note that abstract conformance is computed by first computing the abstract trivial success subsets or supersets of the involved property formulas, and then applying the operators of the abstract domain. For abstract domains which may lose precision with their $(\sqcup)$ abstract operator, more advanced techniques for leveraging multiple abstrac-

1 | **p_nat_nat** := { :- **pred** $_ (X, Y) : \text{nat}(X) \Rightarrow \text{nat}(Y)$. }.

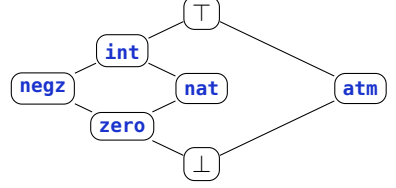
(a) Predicate property.

```

2 | :- pred n2n(X, Y) : nat(X) => nat(Y). % A1
3 | :- pred a2n(X, Y) : atm(X) => nat(Y). % A2
4 | :- pred i2z(X, Y) : int(X) => zero(Y). % A3
5 | :- pred z2i(X, Y) : zero(X) => int(Y). % A4
6 | :- pred nz2n(X, Y) : negz(X) => nat(Y). % A5

```

(b) Assertions.



(c) Abstract domain lattice.

Figure 1: Example case analysis on a predicate property and assertions.

tions become necessary, *e.g.*, *covering* (Debray et al. 1997). We now relate the notions of conformance and abstract conformance.

Theorem 4.1.

Let p be a predicate, Π be a predicate property: $p \prec^{\sharp-} \Pi \Rightarrow p \prec \Pi$, and $p \not\prec^{\sharp+} \Pi \Rightarrow p \not\prec \Pi$.

Proof.

The proofs proceed by contradiction and direct proof, respectively, using Defs. 4.2 to 4.8 and some basic set manipulation. Detailed proofs can be found in Appendix A. $\square$

Example 4.2 (Abstract conformance).

Consider determining conformance to the predicate property in Figure 1a – which, for simplicity, we will interchangeably refer to as Π for the rest of the example – given the assertions $\mathcal{A} = \{A_1, \dots, A_5\}$ in Figure 1b. (Notice that the property formulas of both Π and $\mathcal{A}$ include elements of the abstract domain represented by the lattice in Figure 1c). Their corresponding sets of assertion conditions are:

$$\Pi = \{\text{calls}(_, (X, Y), \text{nat}(X)), \text{success}(_, (X, Y), \text{nat}(X), \text{nat}(Y))\}$$

$$\mathcal{A} = \{\text{calls}(\text{Pred}_i, \text{Pre}_i), \text{success}(\text{Pred}_i, \text{Pre}_i, \text{Post}_i) \mid A_i \in \mathcal{A}\}$$

We aim to determine which predicates partially specified by $\mathcal{A}$ abstractly conform to the predicate property Π . For each predicate Pred_i and its associated **calls** and **success** assertion conditions, we: (1) determine abstract conformance to the anonymous **calls** condition of Π ; (2) determine abstract conformance to the anonymous **success** condition of Π ; and (3) determine abstract conformance to Π :

Tables 1 and 2 summarize the abstract conformance analysis between the **calls** and **success** assertion conditions of each predicate and those of Π , respectively. Specifically, for **calls** (in Table 1), we compare the pre-conditions and apply Definition 4.7; for **success** (in Table 2), we compare both pre- and post-conditions and apply Definition 4.8. As a summary, the only predicate that definitely conforms to **p_nat_nat** is **n2n/2**, since both of its assertion conditions conform to Π .

4.2 Wrappers

Consider a predicate p and a predicate property Π s.t. p can be covered by Π . From Definition 4.4 we know that given their respective pre-conditions Pre and ${}^\circ\text{Pre}$, $\text{Pre}^\sharp \supseteq {}^\circ\text{Pre}^\sharp$. Thus, according to Definition 4.7, p may abstractly conform to Π ($p \prec^{\sharp+} \Pi$), since Pre

Table 1: Abs. conf. on “calls” example with ${}^{\circ}Pre = \mathbf{nat}(X)$.

$Pred_i$	Pre_i	Relation w. ${}^{\circ}Pre$	Abs. Conf.
$n2n(X, Y)$	$\mathbf{nat}(X)$	$\mathbf{nat}(X) = \mathbf{nat}(X)$	yes
$a2n(X, Y)$	$\mathbf{atm}(X)$	$\mathbf{atm}(X) \sqcap \mathbf{nat}(X) = \perp$	no
$i2z(X, Y)$	$\mathbf{int}(X)$	$\mathbf{int}(X) \sqsupseteq \mathbf{nat}(X)$	maybe
$z2i(X, Y)$	$\mathbf{zero}(X)$	$\mathbf{zero}(X) \sqsubseteq \mathbf{nat}(X)$	maybe
$nz2n(X, Y)$	$\mathbf{negz}(X)$	$\mathbf{negz}(X) \sqcap \mathbf{nat}(X) \neq \perp$ $\wedge \mathbf{negz}(X) \not\sqsubseteq \mathbf{nat}(X)$ $\wedge \mathbf{negz}(X) \not\sqsupseteq \mathbf{nat}(X)$	maybe

Table 2: Abs. conf. on “success” example with ${}^{\circ}Pre = \mathbf{nat}(X)$ and ${}^{\circ}Post = \mathbf{nat}(Y)$.

$Pred_i$	Pre_i	Relation w. ${}^{\circ}Pre$	$Post_i$	Relation w. ${}^{\circ}Post$	Abs. Conf.
$n2n(X, Y)$	$\mathbf{nat}(X)$	$\mathbf{nat}(X) = \mathbf{nat}(X)$	$\mathbf{nat}(Y)$	$\mathbf{nat}(Y) = \mathbf{nat}(Y)$	yes
$a2n(X, Y)$	$\mathbf{atm}(X)$	$\mathbf{atm}(X) \sqcap \mathbf{nat}(X) = \perp$	$\mathbf{nat}(Y)$	$\mathbf{nat}(Y) = \mathbf{nat}(Y)$	maybe
$i2z(X, Y)$	$\mathbf{int}(X)$	$\mathbf{int}(X) \sqsupseteq \mathbf{nat}(X)$	$\mathbf{zero}(Y)$	$\mathbf{zero}(Y) \sqsubseteq \mathbf{nat}(Y)$	yes
$z2i(X, Y)$	$\mathbf{zero}(X)$	$\mathbf{zero}(X) \sqsubseteq \mathbf{nat}(X)$	$\mathbf{int}(Y)$	$\mathbf{int}(Y) \sqsupseteq \mathbf{nat}(Y)$	maybe
$nz2n(X, Y)$	$\mathbf{negz}(X)$	$\mathbf{negz}(X) \sqcap \mathbf{nat}(X) \neq \perp$ $\wedge \mathbf{negz}(X) \not\sqsubseteq \mathbf{nat}(X)$ $\wedge \mathbf{negz}(X) \not\sqsupseteq \mathbf{nat}(X)$	$\mathbf{nat}(Y)$	$\mathbf{nat}(Y) = \mathbf{nat}(Y)$	maybe

may describe *more* admissible call states for p than ${}^{\circ}Pre$, which can lead to omitting some run-time check errors that would be raised by ${}^{\circ}Pre$.

Example 4.3 (Weak abstract conformance).

Consider a query `?- foo(even)` to the following program.

```

1 | p_nat := { :- pred _(N) : nat(N). }. % 1-ary predicates for naturals.
2 |
3 | :- pred even(N) : int(N).           :- pred foo(P) : p_nat(P).
4 | even(N) :-                             foo(P) :- P(10). % (1)
5 |   integer(N), 0 is N mod 2.           foo(P) :- P(-10). % (2)

```

Take a derivation of such query that starts by reducing to the body of the first clause (1): no calls assertion condition violation is expected since all predicates that conform to **p_nat** must accept all natural numbers on calls. Now, take a derivation that reduces to the body of the second clause (2): a calls assertion condition violation is expected since all predicates that conform to **p_nat** must raise an error for any input different from a natural number. However, in this particular case, no error is raised, since the predicate `even/1` accepts any integer on calls. Moreover, if we had:

```

6 | p_neg := { :- pred _(N) : neg(N). }. % 1-ary predicates for negatives.
7 |
8 | :- pred bar(P) : p_neg(P).
9 | bar(P) :- P(-4).

```

then a clause like `foo(P) :- bar(P).` would be problematic. For this clause, looking at the assertion of `foo(P)`, the predicate in **P** is required to conform to **p_nat**, which would be an error when calling `bar(P)` since **p_neg** is disjoint from **p_nat**. However, if we consider

the particular case in which $P = \text{even}$, it may not. So, according to Definition 4.7, we could only conclude that $\text{even}/1 \prec^{\#} \mathbf{p_nat}$, that is, $\text{even}/1$ may abstractly conform.

In the example above, we motivate the need for such a restrictive condition for abstract conformance on *calls* (see Definition 4.7). However, we may want to use predicates whose set of admissible calls is greater than that of a predicate property, but without unexpected behavior. To this end, we propose a technique to restrict the set of admissible calls of a predicate p described by Pre to match that of Pre in a program analysis friendly manner. This restriction is implemented using *wrappers*. A wrapper for p with Π is simply a new predicate $w(\bar{v}) :- p(\bar{v})$ with an assertion “ $:- \text{pred } w(\bar{v}) : \text{Pre}.$ ” (note that fields of *pred* assertions, in this case the post-condition, can be omitted, equivalently to *true*). A wrapper for p with Π also makes explicit the intention of creating a Π -tailored version of p . Additionally, wrappers can also be used to alleviate the process of determining abstract conformance on *calls* (particularly useful in the implementation), since the wrapper would syntactically (and thus, semantically) match the pre-condition of the predicate property.

Example 4.4 (Wrapper).

As a follow-up of the previous example, consider wrapping $\text{even}/1$ with $\mathbf{p_nat}$:

```
10 | :- pred even_nat(N) : nat(N).
11 | even_nat(N) :- even(N).
```

Intuitively, $\text{even_nat}/1$ conforms to $\mathbf{p_nat}$, and the analyzer can now infer that the clause $\text{foo}(P) :- \text{bar}(P).$ should raise an error since $\text{even_nat}/1$ only accepts naturals.

4.2.1 Rationale for explicit wrappers

The design of *Hiord*[#] follows the philosophy behind the *Ciao* system (Hermenegildo et al. 2012), which extends *Prolog* with static and dynamic assertion checking (among other modular extensions) without altering its untyped nature. We also considered some alternative solutions to the problem at hand, such as *tainting* each predicate passed as an argument annotated with a *predicate property*, and restricting its future use in all internal (recursive) calls. However, this approach would have required modifying the standard *Prolog* semantics regarding higher-order calls. The use of *wrappers* allows us to simulate this behavior without altering the underlying semantics.

4.3 First-order representation of predicate properties

As mentioned in §2, the abstract interpretation-based static analyzer can *infer* properties about higher-order programs, and also *verify* first-order assertions. However, here we obviously need to deal with predicate properties in assertions. Usually, for a new type of property, a new abstract domain is needed. As an alternative approach, we herein propose representing predicate properties as first-order properties of a kind which can be natively supported by the analyzer, thus allowing us to leverage existing and mature abstract domains. More concretely, we propose representing predicate properties as *regular types*, a special kind of properties (and thus defined as predicates) that are used to describe the *shape* of a term. Intuitively, such types will capture sets of predicate names. For example, given the predicate property $\mathbf{pp}$, we can represent that the predicates p , and q strongly, and r weakly conforms to $\mathbf{pp}$ as the following regular types: $\mathbf{pp}^-/1 = \{\mathbf{pp}^-(p), \mathbf{pp}^-(q)\}$

Algorithm 1 [*Hiord*[#]]: Verify a higher-order program with higher-order assertions

Input: Program: P , assertions: $\mathcal{A}$, abstract queries: $\mathcal{Q}^\#$
Output: Verified status (**checked**/**false**/**check**) for the assertions $\mathcal{A}$ of P : $\mathcal{V}$

```

1:  $R \leftarrow \emptyset$ 
2: repeat  $\triangleright$  start fixpoint computation
3:    $R' \leftarrow R$   $\triangleright$  save state to check fixpoint convergence
4:   for all predicate property  $\Pi \in P$  do
5:      $R \leftarrow R \cup \{\pi^-(p) \mid p \in P \wedge p \prec^{\#-} \Pi\} \cup \{\pi^+(p) \mid p \in P \wedge p \prec^{\#+} \Pi\}$   $\triangleright$  regtypes
6:   end for
7: until  $R = R'$   $\triangleright$  fixpoint reached
8:  $\mathcal{V} \leftarrow \text{acheck}(\mathcal{A}, \llbracket P \cup R \rrbracket_{\mathcal{Q}^\#}^\#)$   $\triangleright$  first-order assertion checking process

```

and $\mathbf{pp}^+/1 = \{\mathbf{pp}^+(p), \mathbf{pp}^+(q), \mathbf{pp}^+(r)\}$.⁷ Formally, given a predicate property Π , we define two associated regular types: $\pi^-/1$ and $\pi^+/1$, that capture the set of predicates that *strongly* and *weakly* abstractly conform to Π as follows: $\pi^-/1 = \{\pi^-(p) \mid p \in P \wedge p \prec^{\#-} \Pi\}$, and $\pi^+/1 = \{\pi^+(p) \mid p \in P \wedge p \prec^{\#+} \Pi\}$. By definition, $\pi^-/1$ is a *subtype* of $\pi^+/1$. These regular types reduce the compile-time checking of higher-order assertions to that of first-order assertions. Regular types can be abstracted and inferred by several abstract domains; for concreteness we use **eterms** (Vaucheret and Bueno 2002).

4.4 *Hiord*[#] algorithm

We now present *Hiord*[#], the core algorithm for the compile-time verification of a higher-order program P with higher-order assertions $\mathcal{A}$ (Algorithm 1). First, it initializes a set of rules R , and it computes the regular type representations of each predicate property Π in P , that is, the π^- and π^+ predicates respectively (lines 4 to 6). This computation is performed by directly applying Definitions 4.7 to 4.9 using the operators of the abstract domain, and (implicitly) extending the program P with R . Since predicate properties can include predicate property literals from other predicate properties – that is, *dependencies* among predicate properties – lines 4 to 6 are repeated until a *fixpoint* is reached (lines 2 and 7). Next, it computes the abstract interpretation of P , augmented with the regular type representations of every predicate property, for the set of abstract queries $\mathcal{Q}^\#$ (line 8). Finally, it performs the compile-time verification of the set of (now first-order) assertions $\mathcal{A}$ w.r.t. the static analysis results, where predicate properties are now treated as standard regular types (line 8). As the result of the algorithm, we obtain the verified status of each assertion of $\mathcal{A}$, where each assertion can be discharged (**checked**), disproved (**false**) and an error flagged, or left in **check** status, and subject to run-time checks, as in Stulova et al. (2014). We argue that, despite the inherent complexity of the verification problem in hand, the proposed concepts make the compile-time checking algorithm clear and concise; and, more importantly, easily implementable using a first-order assertion checker.

⁷ Or, using Ciao’s functional notation: “:- **regtype** $\mathbf{pp}^+/1$. $\mathbf{pp}^+ := p \mid q \mid r..$ ”

5 Implementation and experiments

To demonstrate the potential of our approach, we have implemented a prototype of the *Hiord[#]* technique as part of the *Ciao* system. It implements Algorithm 1 and uses *CiaoPP*, the *Ciao* program preprocessor, with the *eterms* abstract domain. We ran experiments on a set of small but representative higher-order programs that were not possible to verify until this point. We illustrate below our experiments with a selection of these programs.

5.1 A synthetic benchmark

We started by defining a test case comprising a predicate property using an anonymous *pred* assertion and 25 predicates, each with a *pred* assertion, designed to exhaustively cover all possible orderings between the pre- and post-conditions of the predicate property and of each predicate. We then ran *Hiord[#]*, obtaining the correct results that 2 predicates *definitely did conform* and 7 predicates *definitely did not conform*, with 16 predicates left where no definite conclusion could be reached.

5.2 Higher-order list utilities

We defined various partially specified higher-order utility predicates specialized for working with lists of a particular type **t**, for example, **t**(X) :- num(X). For example, consider the **t_cmp** predicate property defined below:

```
1 | t_cmp := { :- pred _ (X,Y) : (t(X), t(Y)). }.
```

which describes *comparator* predicates of elements of type **t**, that we then use in the higher-order assertion for a comparator-parameterizable *quicksort* implementation:

```
2 | :- pred qsort(Xs,P,Ys) : (list(t,Xs), t_cmp(P)) => list(t,Ys).
```

and where the **partition/4** predicate includes a call **P**(X,Y). The analysis is able to propagate the **t_cmp** property on **P** to that point, and if in **P**(X,Y), **X** is inferred to be bound to, for example, **a**, an error is statically captured by *Hiord[#]*. Consider the following comparators:

<pre>3 :- pred lex(X,Y) : (term(X), term(Y)). 4 lex(X,Y) :- X @< Y.</pre>	<pre>5 :- pred lex_t(X,Y) : (t(X), t(Y)). 6 lex_t(X,Y) :- lex(X,Y).</pre>
---	--

For a query **?- qsort(Xs,lex,Ys)**, *Hiord[#]* reports a warning on *calls* since **lex/2** $\prec^{\#} \mathbf{t_cmp}$ (it weakly abstractly conforms). Intuitively, **lex/2** is not definitely conformant since it will not raise a run-time check error when called with a term that is not of type **t**, introducing unexpected behavior. For a query **?- qsort(Xs,lex_t,Ys)** *Hiord[#]* proves that it behaves correctly w.r.t. its higher-order assertion, since **lex_t/2** $\prec^{\#} \mathbf{t_cmp}$.

Additionally, consider a predicate property which represents a parameterizable sorter of lists of elements of type **t**, defined “in terms of” the **t_cmp** predicate property:

```
7 | t_sort := { :- pred _ (Xs,C,Ys) : (list(t,Xs), t_cmp(C)) => list(t,Ys). }.
```

For determining that **qsort/3** $\prec^{\#} \mathbf{t_sort}$, *Hiord[#]* would need to perform an additional iteration of the fixpoint computation after the one above, that is, after computing the predicates that weakly or strongly abstractly conform to the **t_cmp** predicate property.

5.3 HTTP server

Consider the following schematic HTTP server, parameterized by a predicate that must be able to handle four REST operations. We use regular types for representing requests and responses, and a predicate property for representing handlers:

```

1 | handler := { :- pred _ (Rq,Rs) : req(Rq) => res(Rs) . }.
2
3 | :- regtype req/1. req := 'DELETE' | 'GET' | 'POST' | 'PUT'.
4 | :- regtype res/1. res := 'OK' | 'CREATED' | 'BAD_REQUEST' | 'NOT_FOUND'.

```

and we add the following higher-order assertion to the server predicate:

```

5 | :- pred server(H,Rq,Rs) : (handler(H), req(Rq)) => res(Rs).

```

Hiord[#] detects that the predicate `h/2` does not definitely conform to `handler` (`h/2` $\prec^{#+}$ `handler`) due to one its clauses:

```

6 | h('PUT', Rs) :- ..., Rs = 'BAD_REQ'. % Bug, should be 'BAD_REQUEST'

```

5.4 Dutch national flag

This problem involves sorting a list of red, white, or blue elements, such that elements of the same color are grouped together in a specified order (typically red, then white, then blue). However, we want to generalize the solution by allowing the user to provide a comparator that, given two elements, yields their comparison. We first define regular types to represent colored elements and the result of their comparison:

```

1 | :- regtype rwb/1. rwb := r | w | b.
2 | :- regtype lge/1. lge := < | > | = .

```

Next, we define a `dutch_cmp` predicate property describing comparators between `rwb` elements; and provide a higher-order assertion to the `dutch_flag/3` higher-order predicate:

```

3 | dutch_cmp := { :- pred _ (X,R,Y) : (rwb(X), rwb(Y)) => lge(R) . }.
4 | :- pred dutch_flag(C,Xs,Ys) : (dutch_cmp(C), list(rwb,Xs)) => list(rwb,Ys).

```

Assume that we are given the implementation of `dutch_flag/3` and we need to provide a comparator `cmp/3` which conforms to `dutch_cmp`. Consider a first implementation attempt:

```

6 | cmp(red,=, red). cmp(white,=,white). cmp(blue,=, blue).
7 | cmp(red,<,white). cmp(white,>, red). cmp(blue,>, red).
8 | cmp(red,<, blue). cmp(white,<, blue). cmp(blue,>,white).
1 | :- regtype rt1/1.
2 | rt1 := red | blue
3 |      | white.

```

When determining its conformance to `dutch_cmp`, *Hiord*[#] finds that `cmp/3` $\not\prec^{#+}$ `dutch_cmp`, since CiaoPP infers the regular type `rt1` for the elements to compare, and `rt1` $\sqcap$ `rwb` = $\perp$ in `eterms`. We proceed by correcting it, but we accidentally mistype some of the `r` elements for `o` elements in lines 7 and 8; and CiaoPP infers the following assertion and regular type:

```

6 | :- pred cmp(X,R,Y) : (rt2(X), rwb(Y)) => lge(R).
7 | cmp(o,=,r). cmp(w,=,w). cmp(b,=,b).
8 | cmp(o,<,w). cmp(w,>,r). cmp(b,>,r).
9 | cmp(r,<,b). cmp(w,<,b). cmp(b,>,w).
1 | :- regtype rt2/1.
2 | rt2 := r | b
3 |      | w | o.

```

However, *Hiord*[#] now reports that `cmp/3` $\prec^{#+}$ `dutch_cmp`, since it would not raise a run-time check error when called with an `o` on its first argument. Formally, $(rt2(X), rwb(Y)) \sqsubseteq (rwb(X), rwb(Y))$ in `eterms`. In an attempt at improving the precision of the ordering, we refine `cmp/3` to yield more informative results on the order relation between elements:

<pre> 6 :- pred cmp(X,R,Y) : (rwb(X), rwb(Y)) => lgLGe(R). 7 cmp(r, =, r). cmp(w, =, w). cmp(b, =, b). 8 cmp(r, <, w). cmp(w, >, r). cmp(b, >>, r). 9 cmp(r, <<, b). cmp(w, <, b). cmp(b, >, w). </pre>	<pre> 10 :- regtype lgLGe/1. 11 lgLGe := < > 12 << >> 13 = . </pre>
--	---

In particular, we introduce `<<` and `>>` to reflect that X is “much lower” than Y , and vice-versa; and define a new regular type and assertion. However, *Hiord*[#] still reports that `cmp/3` $\prec^{\#+}$ **dutch_cmp**, since `cmp/3` may yield comparison results that are not reflected in **lge**. Formally $\text{lgLGe}(R) \sqsupseteq \text{lge}(R)$ in *eterns*. Finally, we develop the following comparator:

```

6 | :- pred cmp(X, R, Y) : (rwb(X), rwb(Y)) => lge(R).
7 | cmp(r, =, r). cmp(w, =, w). cmp(b, =, b).
8 | cmp(r, <, w). cmp(w, >, r). cmp(b, >, r).
9 | cmp(r, <, b). cmp(w, <, b). cmp(b, >, w).

```

And *Hiord*[#] proves that `cmp/3` $\prec^{\#-}$ **dutch_cmp**, since it behaves exactly as expected.

6 Conclusions

We have presented *Hiord*[#], a novel approach for the compile-time verification of higher-order (C)LP programs with higher-order assertions. We started by refining both the syntax and semantics of predicate properties. Then, we introduced an abstract criterion to determine whether a predicate conforms with a predicate property at compile time. We also motivated and explained a *wrapper*-based technique for “casting” predicate usage in a program analysis-friendly manner that enhances and complements the proposed abstract criterion. We then proposed a technique for dealing with these properties using an abstract interpretation-based static analyzer for programs with first-order assertions. Finally, we reported on a prototype implementation and studied the effectiveness of the approach with various examples within the *Ciao* system. We believe our proposal constitutes a practical approach to closing the existing gap in the verification at compile time of higher-order assertions; and that it is quite general and flexible, and can be applied, at least conceptually, to other similar gradual approaches.

Competing interests

The authors declare none.

References

- CABEZA, D., HERMENEGILDO, M., AND LIPTON, J. *Hiord: A Type-Free Higher-Order Logic Programming Language with Predicate Abstraction*. In *ASIAN'04* 2004, number 3321 in LNCS, pp. 93–108. Springer-Verlag.
- CARDELLI, L. *Typeful programming*. In NEUHOLD, E. J. AND PAUL, M., editors, *Formal Description of Programming Concepts* 1989, IFIP State-of-the-Art Reports. Springer.
- COUSOT, P. AND COUSOT, R. *Abstract Interpretation: a Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints*. In *Proc. of POPL'77* 1977, pp. 238–252. ACM Press.
- DEBRAY, S., LOPEZ-GARCIA, P., AND HERMENEGILDO, M. *Non-Failure Analysis for Logic Programs*. In *1997 International Conference on Logic Programming* 1997, pp. 48–62, Cambridge, MA. MIT Press, Cambridge, MA.

- HERMENEGILDO, M., BUENO, F., CARRO, M., LOPEZ-GARCIA, P., MERA, E., MORALES, J., AND PUEBLA, G. 2012. An Overview of Ciao and its Design Philosophy. *TPLP*, 12, 1–2, 219–252.
- HERMENEGILDO, M., PUEBLA, G., AND BUENO, F. Using Global Analysis, Partial Specifications, and an Extensible Assertion Language for Program Validation and Debugging. In *The Logic Programming Paradigm: a 25-Year Perspective* 1999, pp. 161–192. Springer-Verlag.
- HERMENEGILDO, M., PUEBLA, G., BUENO, F., AND GARCIA, P. L. 2005. Integrated Program Debugging, Verification, and Optimization Using Abstract Interpretation (and The Ciao System Preprocessor). *Science of Computer Programming*, 58, 1–2, 115–140.
- HILL, P. AND LLOYD, J. 1994. *The Goedel Programming Language*. MIT Press, Cambridge MA.
- KÖRNER, P., LEUSCHEL, M., BARBOSA, J., SANTOS-COSTA, V., DAHL, V., HERMENEGILDO, M. V., MORALES, J. F., WIELEMAKER, J., DIAZ, D., ABREU, S., AND CIATTO, G. 2022. Fifty Years of Prolog and Beyond. *TPLP, 20th Anniversary Special Issue*, 22, 6, 776–858.
- MARLOW, S., editor 2010. *Haskell 2010 Language Report*. Haskell Language Committee.
- MILLER, D. 1991. A logic programming language with λ -abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4), 497–536.
- MUTHUKUMAR, K. AND HERMENEGILDO, M. 1992. Compile-time Derivation of Variable Dependency Using Abstract Interpretation. *Journal of Logic Programming*, 13, 2/3, 315–347.
- PUEBLA, G., BUENO, F., AND HERMENEGILDO, M. An Assertion Language for Constraint Logic Programs. In *Analysis and Visualization Tools for Constraint Programming* 2000a, number 1870 in LNCS, pp. 23–61. Springer-Verlag.
- PUEBLA, G., BUENO, F., AND HERMENEGILDO, M. Combined Static and Dynamic Assertion-Based Debugging of Constraint Logic Programs. In *Proc. of LOPSTR’99* 2000b, LNCS 1817, pp. 273–292. Springer-Verlag.
- SANCHEZ-ORDAZ, M., GARCIA-CONTRERAS, I., PEREZ-CARRASCO, V., MORALES, J. F., LOPEZ-GARCIA, P., AND HERMENEGILDO, M. 2021. VeriFly: On-the-fly Assertion Checking via Incrementality. *Theory and Practice of Logic Programming*, 21, 6, 768–784.
- SIEK, J. G. AND TAHA, W. Gradual Typing for Functional Languages. In *Scheme and Functional Programming Workshop* 2006, pp. 81–92. University of Chicago Department of Computer Science.
- SOMOGYI, Z., HENDERSON, F., AND CONWAY, T. 1996. The Execution Algorithm of Mercury: an Efficient Purely Declarative Logic Programming Language. *Journal of Logic Programming*, 29, 1–3, 17–64.
- STULOVA, N., MORALES, J., AND HERMENEGILDO, M. Assertion-based Debugging of Higher-Order (C)LP Programs. In *PPDP’14* 2014, pp. 225–235. ACM.
- STULOVA, N., MORALES, J. F., AND HERMENEGILDO, M. 2018. Some Trade-offs in Reducing the Overhead of Assertion Run-time Checks via Static Analysis. *Science of Computer Programming*, 155, 3–26.
- VAUCHERET, C. AND BUENO, F. More Precise yet Efficient Type Inference for Logic Programs. In *SAS’02* 2002, number 2477 in LNCS, pp. 102–116. Springer.

Appendix A Proof of Theorem 4.1

We start with some auxiliary lemmas.

Lemma A.1.

Given two property formulas F_1 and F_2 , $(F_1 \wedge F_2)^\sharp = F_1^\sharp \cap F_2^\sharp$.

Proof.

We proceed by direct proof. Assume $(F_1 \wedge F_2)^\sharp \neq \emptyset$, take any $\theta \in (F_1 \wedge F_2)^\sharp$. By Definition 4.1, $\theta \Rightarrow_P (F_1 \wedge F_2)$, and from Definition 2.1, $\theta \Rightarrow_P F_1 \wedge \theta \Rightarrow_P F_1$. Hence, by Definition 4.1, $\theta \in F_1^\sharp \wedge \theta \in F_2^\sharp$, and from the properties of set intersection ($\cap$), we conclude $\theta \in (F_1^\sharp \cap F_2^\sharp)$. Assume now $(F_1 \wedge F_2)^\sharp = \emptyset$, then $\forall \theta \in F_1^\sharp. \theta \notin F_2^\sharp$; hence $F_1^\sharp \cap F_2^\sharp = \emptyset$. $\square$

Lemma A.2.

Given the calls assertion condition $\text{calls}(p(\bar{v}), \text{Pre}) \in \mathcal{A}$, and the anonymous calls assertion condition $\text{calls}(_)(\bar{v}), \text{Pre}) \in \Pi$ associated to Π :

$$\text{Pre}^{\sharp+} \sqsubseteq \text{Pre}^{\sharp-} \Rightarrow p \text{ can be covered with } \Pi.$$

Proof.

The proof proceeds by assuming $\text{Pre}^{\sharp+} \sqsubseteq \text{Pre}^{\sharp-}$ and showing $\text{Pre}^\sharp \subseteq \text{Pre}^\sharp$.

$$\begin{aligned} & \text{Pre}^{\sharp+} \sqsubseteq \text{Pre}^{\sharp-} \\ \Leftrightarrow & \gamma(\text{Pre}^{\sharp+}) \subseteq \gamma(\text{Pre}^{\sharp-}) && \text{\textit{[Abstract operator ($\sqsubseteq$)]}} \\ \Leftrightarrow & \text{Pre}^\sharp \subseteq \gamma(\text{Pre}^{\sharp+}) \subseteq \gamma(\text{Pre}^{\sharp-}) \subseteq \text{Pre}^\sharp && \text{\textit{[Defs. 4.2 and 4.3]}} \\ \Leftrightarrow & \text{Pre}^\sharp \subseteq \text{Pre}^\sharp && \text{\textit{[Partial order ($\subseteq$)]}} \end{aligned}$$

$\square$

We now proceed to the proof of Theorem 4.1.

Theorem 4.1.

Let p be a predicate, Π be a predicate property: $p \prec^{\sharp-} \Pi \Rightarrow p \prec \Pi$, and $p \not\prec^{\sharp+} \Pi \Rightarrow p \not\prec \Pi$.

Proof.

Let P be a program, p be a predicate s.t. $p \in P$, and Π be a predicate property. We will prove each case separately.

- We proceed by contradiction. Assume that p abstractly conforms to Π :

$$p \prec^{\sharp-} \Pi \tag{A.1}$$

and that p does not conform to Π :

$$p \not\prec \Pi \tag{A.2}$$

First, by Equation (A.1) and Lemma A.2, we know that p can be *covered* with Π , hence

$$\text{Pre}^\sharp \subseteq \text{Pre}^\sharp \tag{A.3}$$

From Equation (A.2) we know that there exists a query $Q_p = (p(\bar{v}), \theta_0)$ to p s.t. Π is *not* redundant for p . Thus, by negating the redundancy condition from Definition 4.5, we know that: $\exists D' \in \text{derivs}_{\mathcal{A}'}(P, Q_p)$. $D'_{[-1]} = \langle G' \mid \theta \mid \{\ell'\} \rangle$, and

$\exists D \in \text{derivs}_{\mathcal{A}}(P, Q_p)$. $\mathcal{U}(D) = \mathcal{U}(D')$, s.t. $D_{[-1]} \rightsquigarrow_{\mathcal{A}}^* \langle G | \theta | \emptyset \rangle$ through any derivation that reduces only **check** literals; where $\mathcal{A}'$ is defined as in Definition 4.6. We also know that both $\text{derivs}_{\mathcal{A}'}(P, Q_p)$ and $\text{derivs}_{\mathcal{A}}(P, Q_p)$ share the same initial state $S = \langle p(\bar{v}) | \theta_0 | \emptyset \rangle$ given that they are deriving the same query Q_p to p . We now consider two cases: the error comes from a **calls** or **success** assertion condition in $\mathcal{A}'$.

Calls: The label ℓ' identifies a **calls** assertion condition C' in $\mathcal{A}'$ – of the form $\text{calls}(p(\bar{v}), \text{Pre} \wedge {}^\circ\text{Pre})$ (from the definition of $\mathcal{A}'$) – which fails to be checked for θ ; and the **calls** assertion condition C in $\mathcal{A}$ – of the form $\text{calls}(p(\bar{v}), \text{Pre})$ – is checked for θ . On the one hand:

$$\begin{aligned}
& C' \text{ fails to be checked for } \theta \\
& \Leftrightarrow \theta \not\Rightarrow_P \text{Pre} \wedge {}^\circ\text{Pre} && \{\text{Semantics with assertions}\} \\
& \Leftrightarrow \theta \notin (\text{Pre} \wedge {}^\circ\text{Pre})^\natural && \{\text{Def. 4.1}\} \\
& \Leftrightarrow \theta \notin \text{Pre}^\natural \cap {}^\circ\text{Pre}^\natural && \{\text{Lem. A.1}\} \\
& \Leftrightarrow \theta \notin {}^\circ\text{Pre}^\natural && \{\text{Eq. (A.3)}\}
\end{aligned}$$

On the other hand:

$$\begin{aligned}
& C \text{ is checked for } \theta \\
& \Leftrightarrow \theta \Rightarrow_P \text{Pre} && \{\text{Semantics with assertions}\} \\
& \Leftrightarrow \theta \in \text{Pre}^\natural && \{\text{Def. 4.1}\}
\end{aligned}$$

Since $p \prec^{\sharp-} \Pi$ (by Equation (A.1)), we know from Definition 4.7 that:

$$\begin{aligned}
& \text{Pre}^{\sharp+} \sqsubseteq {}^\circ\text{Pre}^{\sharp-} \\
& \Leftrightarrow \gamma(\text{Pre}^{\sharp+}) \subseteq \gamma({}^\circ\text{Pre}^{\sharp-}) && \{\text{Abstract operator } (\sqsubseteq)\} \\
& \Leftrightarrow \text{Pre}^\natural \subseteq \gamma(\text{Pre}^{\sharp+}) \subseteq \gamma({}^\circ\text{Pre}^{\sharp-}) \subseteq {}^\circ\text{Pre}^\natural && \{\text{Defs. 4.2 and 4.3}\}
\end{aligned}$$

Since $\theta \in \text{Pre}^\natural$ and $\sqsubseteq$ is a partial order, it implies that $\theta \in {}^\circ\text{Pre}^\natural$, contradicting $\theta \notin {}^\circ\text{Pre}^\natural$. $\bot$

Success: The label ℓ' identifies a **success** assertion condition C' in $\mathcal{A}'$ – of the form $\text{success}(p(\bar{v}), {}^\circ\text{Pre}, {}^\circ\text{Post})$ – which fails to be checked for θ ; and all **success** assertion conditions C_i in $\mathcal{A}$ – of the form $\text{success}(p(\bar{v}), \text{Pre}_i, \text{Post}_i)$ with $i \in 1..n$ – is checked for θ . On the one hand:

$$\begin{aligned}
& C' \text{ fails to be checked for } \theta \\
& \Leftrightarrow (\theta_0 \Rightarrow_P {}^\circ\text{Pre}) \wedge (\theta \not\Rightarrow_P {}^\circ\text{Post}) && \{\text{Semantics with assertions}\} \\
& \Leftrightarrow (\theta_0 \in {}^\circ\text{Pre}^\natural) \wedge (\theta \notin {}^\circ\text{Post}^\natural) && \{\text{Def. 4.1}\}
\end{aligned}$$

Since $p \prec^{\sharp-} \Pi$ (by Equation (A.1)), we know from Definition 4.8 that there exists a proper subset of **success** assertion conditions $S \subset \mathcal{A}$ s.t.:

$$(\text{Pre}_{\sqcup}^{\sharp-} = \sqcup \{ \text{Pre}^{\sharp-} \mid \text{success}(p(\bar{v}), \text{Pre}, _) \in S \}) \sqsupseteq {}^\circ\text{Pre}^{\sharp+} \quad (\text{A.4})$$

and

$$(\text{Post}_{\sqcup}^{\sharp+} = \sqcup \{ \text{Post}^{\sharp+} \mid \text{success}(p(\bar{v}), _, \text{Post}) \in S \}) \sqsubseteq {}^\circ\text{Post}^{\sharp-} \quad (\text{A.5})$$

From Equations (A.4) and (A.5) and Definitions 4.2 and 4.3, we obtain the following relations:

$$\cup\{Pre^{\sharp} \mid \text{success}(p(\bar{v}), Pre, _) \in S\} \supseteq \gamma(Pre_{\sqcup}^{\sharp-}) \supseteq \gamma(^{\circ}Pre^{\sharp+}) \supseteq ^{\circ}Pre^{\sharp} \quad (\text{A.6})$$

and

$$\cup\{Post^{\sharp} \mid \text{success}(p(\bar{v}), _, Post) \in S\} \subseteq \gamma(Post_{\sqcup}^{\sharp+}) \subseteq \gamma(^{\circ}Post^{\sharp-}) \subseteq ^{\circ}Post^{\sharp} \quad (\text{A.7})$$

Finally, since $\theta_0 \in ^{\circ}Pre^{\sharp}$:

$$\begin{aligned} & \theta_0 \in ^{\circ}Pre^{\sharp} \\ \Leftrightarrow & \theta_0 \in \cup\{Pre^{\sharp} \mid \text{success}(p(\bar{v}), Pre, _) \in S\} && \{\text{Eq. (A.6)}\} \\ \Leftrightarrow & \exists(C = \text{success}(p(\bar{v}), Pre, Post)) \in S. \theta_0 \in Pre^{\sharp} && \{\text{Set union } (\cup)\} \\ \Leftrightarrow & \theta \Rightarrow_P Post && \{C \text{ checked for } \theta, \text{ hypothesis}\} \\ \Leftrightarrow & \theta \in Post^{\sharp} && \{\text{Def. 4.1}\} \\ \Leftrightarrow & \theta \in \cup\{Post^{\sharp} \mid \text{success}(p(\bar{v}), _, Post) \in S\} && \{\text{Set union } (\cup)\} \\ \Leftrightarrow & \theta \in ^{\circ}Post^{\sharp} && \{\text{Eq. (A.7)}\} \end{aligned}$$

which leads to a contradiction with our initial hypothesis $\theta \notin ^{\circ}Post^{\sharp}$. $\sharp$

- We proceed by direct proof. Since p does not abstractly conform to Π :

$$p \not\prec^{\sharp+} \Pi \quad (\text{A.8})$$

we consider two cases: p does not abstractly conform to Π 's anonymous calls assertion condition, or to some anonymous success assertion condition of Π .

Calls: Let $(^{\circ}C = \text{calls}(_(\bar{v}), ^{\circ}Pre)) \in \Pi$, we know that $p \not\prec^{\sharp+} ^{\circ}C$, then:

$$\begin{aligned} & p \not\prec^{\sharp+} ^{\circ}C \\ \Leftrightarrow & Pre^{\sharp+} \sqcap ^{\circ}Pre^{\sharp+} = \perp && \{\text{Def. 4.7}\} \\ \Leftrightarrow & Pre^{\sharp} \sqcap ^{\circ}Pre^{\sharp} = \emptyset && \{\text{Def. 4.3}\} \\ \Leftrightarrow & \forall \theta \in ^{\circ}Pre^{\sharp}. \theta \notin Pre^{\sharp} && \{\text{Set intersection } (\sqcap)\} \\ \Rightarrow & ^{\circ}Pre^{\sharp} \not\subseteq Pre^{\sharp} && \{\text{Proper subset } (\subset)\} \\ \Rightarrow & p \text{ cannot be covered with } \Pi && \{\text{Def. 4.4}\} \\ \Rightarrow & p \not\prec \Pi && \{\text{Defs. 4.5 and 4.6}\} \end{aligned}$$

Success: Let $(^{\circ}C = \text{success}(_(\bar{v}), ^{\circ}Pre, ^{\circ}Post)) \in \Pi$, since $p \not\prec^{\sharp+} ^{\circ}C$, we know that there exists a success assertion condition $(C = \text{success}(p(\bar{v}), Pre, Post)) \in \mathcal{A}$ s.t. the following hold:

$$Pre^{\sharp+} \sqsubseteq ^{\circ}Pre^{\sharp-} \quad (\text{A.9})$$

$$Post^{\sharp+} \sqcap ^{\circ}Post^{\sharp+} = \perp \quad (\text{A.10})$$

$$\exists \theta_0 \in Pre^{\sharp}. \mathcal{S}_{\mathcal{A}}(p(\bar{v}), \theta_0, P, \gamma(Q_p^{\sharp})) \neq \emptyset \quad (\text{A.11})$$

where $\gamma(Q_p^{\sharp})$ is a *superset* of the set of valid queries to p described by p 's calls assertion condition in $\mathcal{A}$. First, by Definitions 4.2 and 4.3, from Equation (A.9)

and Equation (A.10) we obtain:

$$Pre^{\sharp} \subseteq {}^{\circ}Pre^{\sharp} \quad (\text{A.12})$$

$$Post^{\sharp} \cap {}^{\circ}Post^{\sharp} = \emptyset \quad (\text{A.13})$$

Next, from Equation (A.11) and Equation (A.12), it follows that $\theta_0 \in Pre^{\sharp}$ and $\theta_0 \in {}^{\circ}Pre^{\sharp}$. Also, from Equation (A.11) and the definition of the *success context*, we know that

$$\theta \in \mathcal{S}_{\mathcal{A}}(p(\bar{v}), \theta_0, P, \gamma(\mathcal{Q}_p^{\sharp}))$$

hence $\theta \in Post^{\sharp}$. However, by Equation (A.13) we know that $\theta \notin {}^{\circ}Post^{\sharp}$. By Definition 4.1, this means that, on the one hand:

$$\theta_0 \Rightarrow_P Pre \wedge \theta \Rightarrow_P Post \quad (\text{A.14})$$

And in the other hand:

$$\theta_0 \Rightarrow_P {}^{\circ}Pre \wedge \theta \not\Rightarrow_P {}^{\circ}Post \quad (\text{A.15})$$

The above conditions Equation (A.14) and Equation (A.15), together with the definition of the operational semantics with assertions, implies that there exists a query $Q_p = (p(\bar{v}), \theta_0)$ to p s.t. Π is *not* redundant for p . And, by the definition of *non conformance* (Definition 4.6), this implies that p *does not* conform to Π . $\square$